



Department of Electrical & Computer Engineering

ECCE5214: Advanced Logic & Computer Interfacing

Matrix Multiplication Accelerator on Intel Cyclone V SoC FPGA

Final Project Report

Team Members

Name	ID
Salim Amer Al Sulaimi	140441
Maher Salim Al Busaidi	142850
Abdullah Ibrahim Al Husaini	141498

Abstract

This report presents the design, implementation, and evaluation of a matrix multiplication accelerator on an Intel Cyclone V SoC FPGA platform. The project explores two FPGA hardware architectures: an Output-Stationary Systolic Array and an Output-Stationary Fully Parallel architecture. The FPGA implementations are compared with matrix multiplication executed on the ARM Hard Processor System (HPS), including an ARM NEON-optimized software baseline. The system supports flexible matrix dimensions up to 8×8 , where input matrices are provided through JSON files in a Linux-based HPS environment. The HPS parses the JSON input, transfers matrices to the FPGA through memory-mapped Programmed I/O (PIO), starts the accelerator, reads the result, and reports the computation cycles.

Experimental results show that, for an 8×8 matrix multiplication, the Output-Stationary Systolic Array completed the computation in 24 FPGA cycles, while the Output-Stationary Fully Parallel architecture completed it in 8 FPGA cycles. At a 50 MHz FPGA clock, these correspond to 480 ns and 160 ns of raw FPGA compute latency, respectively. Compared with the ARM NEON-optimized software baseline of 3308.672 ns, the systolic FPGA architecture achieved approximately 6.9x raw compute speedup, while the fully parallel FPGA architecture achieved approximately 20.7x raw compute speedup. However, full system timing analysis showed that the PIO-based data transfer interface dominates total execution time, reaching hundreds of microseconds. This confirms that the FPGA core is computationally fast, but the current communication method limits system-level performance. Therefore, a future DDR/DMA-based transfer architecture is recommended for high-performance applications.

Keywords: FPGA acceleration, Cyclone V SoC, matrix multiplication, systolic array, HPS, PIO, ARM NEON, DMA

Table of Contents

Abstract	2
Executive Summary	4
1. Introduction	4
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	6
1.4 Scope	6
2. System Architecture	6
2.1 Overview	6
2.2 HPS-FPGA Communication	6
2.3 High-Level Block Diagram	7
2.4 System Operation	7
3. Hardware Architectures	7
3.1 Output-Stationary Systolic Array	7
3.2 Output-Stationary Fully Parallel Architecture	8
4. Software Implementation	8
5. Verification and Testing	8
6. Results and Discussion	9
6.1 Timing Summary	9
6.2 Transfer Breakdown	9
6.3 Correctness Output	10
6.4 Resource Mapping and Fabric Utilization	10
6.5 Discussion	11
7. Conclusion	11
8. Future Work	12
References	12
Appendix A. Timing Evidence	13
Appendix B. Implementation Screenshots	14

Executive Summary

This report documents an end-to-end SoC FPGA accelerator project for matrix multiplication. The system was built to improve embedded matrix multiplication performance by offloading the compute-intensive operation from the ARM processor to custom FPGA hardware.

The Intel Cyclone V SoC combines a Hard Processor System with FPGA fabric, allowing software and hardware logic to work together in a single platform. In this project, the HPS manages high-level application tasks, including reading JSON input files, configuring hardware control registers, starting execution, and collecting results. The FPGA fabric implements the matrix multiplication accelerator.

Two output-stationary architectures are evaluated: a systolic-array architecture and a fully parallel architecture. The Fully Parallel FPGA architecture achieved the strongest raw computation performance, with a measured 20.7x speedup over the ARM NEON baseline. However, the total system performance was limited by PIO data movement overhead, showing that the communication interface is a major factor in hardware accelerator performance.

1. Introduction

1.1 Background

Matrix multiplication is a fundamental operation in scientific computing, signal processing, machine learning, graphics processing, and embedded acceleration. It requires many repeated multiply-and-accumulate operations, which makes it a suitable candidate for hardware acceleration.

Systolic-array architectures are commonly used for matrix multiplication because they arrange computation into regular processing elements with local communication and predictable data movement. This organization is efficient for VLSI and FPGA implementation because multiple processing elements can operate in parallel [3].

This project applies the systolic-array concept in a modern SoC FPGA environment. The ARM-based HPS handles software control, file input, and system coordination, while the FPGA fabric performs the compute-intensive matrix multiplication operation.

1.2 Problem Statement

A software-only implementation of matrix multiplication on an embedded processor can become a performance bottleneck when matrix sizes increase or when repeated multiplications are required. ARM NEON improves software performance by using Advanced SIMD instructions, but it still operates within the limits of processor-based execution [4].

In addition, transferring matrix data one element at a time through memory-mapped PIO registers introduces significant communication overhead. The lightweight HPS-to-FPGA bridge is appropriate for simple control and status access, but it is not intended for high-throughput bulk data movement [1]. Therefore, although PIO is simple and effective for prototyping, it is not ideal for transferring large matrices between the HPS and FPGA.

For this reason, a higher-performance design is needed in which the FPGA performs matrix multiplication in parallel while the communication overhead between the HPS and FPGA is minimized.

1.3 Objectives

1. Design a custom FPGA accelerator for matrix multiplication.
2. Implement and compare two output-stationary hardware architectures: an Output-Stationary Systolic Array and an Output-Stationary Fully Parallel architecture.
3. Integrate the accelerator with the HPS using Platform Designer.
4. Provide software-visible control through PIO registers.
5. Support flexible matrix dimensions up to 8 x 8.
6. Use JSON files on the HPS to provide input matrix data.
7. Verify FPGA results against a software reference implementation.
8. Compare FPGA performance with an ARM NEON-optimized software baseline.
9. Analyze the effect of communication overhead on total system performance.
10. Identify future improvements, especially DMA-based data transfer.

1.4 Scope

This report focuses on matrix multiplication using integer or fixed-width arithmetic on an Intel Cyclone V SoC FPGA platform. The project includes hardware architecture design, RTL implementation, HPS software development, system integration, correctness verification, performance measurement, and discussion of resource and communication limitations.

The scope does not include floating-point optimization, cache-coherent accelerator protocols, full Linux kernel driver development, or a complete DMA engine implementation. These topics are considered possible future improvements.

2. System Architecture

2.1 Overview

The system is based on the Intel Cyclone V SoC FPGA platform. The HPS runs Linux software and manages application-level tasks, while the FPGA fabric implements the custom matrix multiplication accelerator.

The design follows a hardware/software co-design approach. The HPS is used for flexible control tasks, including file parsing and accelerator configuration. The FPGA fabric is used for parallel computation. This separation allows the system to combine software flexibility with hardware acceleration.

Intel Avalon interfaces are commonly used in FPGA systems for connecting components that read and write registers, access memory, and communicate in Platform Designer systems [2]. In this project, memory-mapped control registers are used to allow the HPS to communicate with the FPGA accelerator.

2.2 HPS-FPGA Communication

The HPS communicates with the FPGA accelerator through memory-mapped interfaces. PIO registers are used to send control signals, matrix dimensions, and input/output data between the processor and the FPGA. These registers are accessed from Linux using `/dev/mem` and `mmap()`.

Port Name	Direction	Width	Used Bits	Function
<code>clk</code>	Input	1 bit	1 bit	System clock for the FPGA matrix accelerator.
<code>reset_n</code>	Input	1 bit	1 bit	Active-low reset. When <code>reset_n = 0</code> , the accelerator resets internal registers, counters, and status signals.
<code>matrix_ctrl</code>	Input	32 bits	Bit 0, Bit 1	Control register from HPS to FPGA. Used to start the accelerator and select the mode.
<code>matrix_dims</code>	Input	32 bits	Bits [3:0], [7:4], [11:8]	Stores matrix dimensions <code>M</code> , <code>K</code> , and <code>N</code> .
<code>matrix_write_addr</code>	Input	32 bits	Lower bits used	Address used by HPS to select where matrix input data is written.
<code>matrix_write_data</code>	Input	32 bits	Usually bits [7:0]	Matrix input value written from HPS to FPGA.
<code>matrix_write_en</code>	Input	32 bits	Bit 0	Write-enable signal. A pulse on bit 0 writes <code>matrix_write_data</code> into the selected matrix address.
<code>matrix_index</code>	Input	6 bits	Bits [5:0]	Selects which output matrix element <code>C[index]</code> should be read. Supports 64 values: 0 to 63.
<code>matrix_status</code>	Output	32 bits	Bits 0-3	Status register from FPGA to HPS. Shows busy, done, error, and mode information.
<code>matrix_data</code>	Output	32 bits	Usually bits [15:0] or [31:0]	Output data register. Returns the selected result matrix value <code>C[index]</code> .
<code>matrix_cycles</code>	Output	32 bits	All 32 bits	Number of FPGA clock cycles used for the matrix computation.

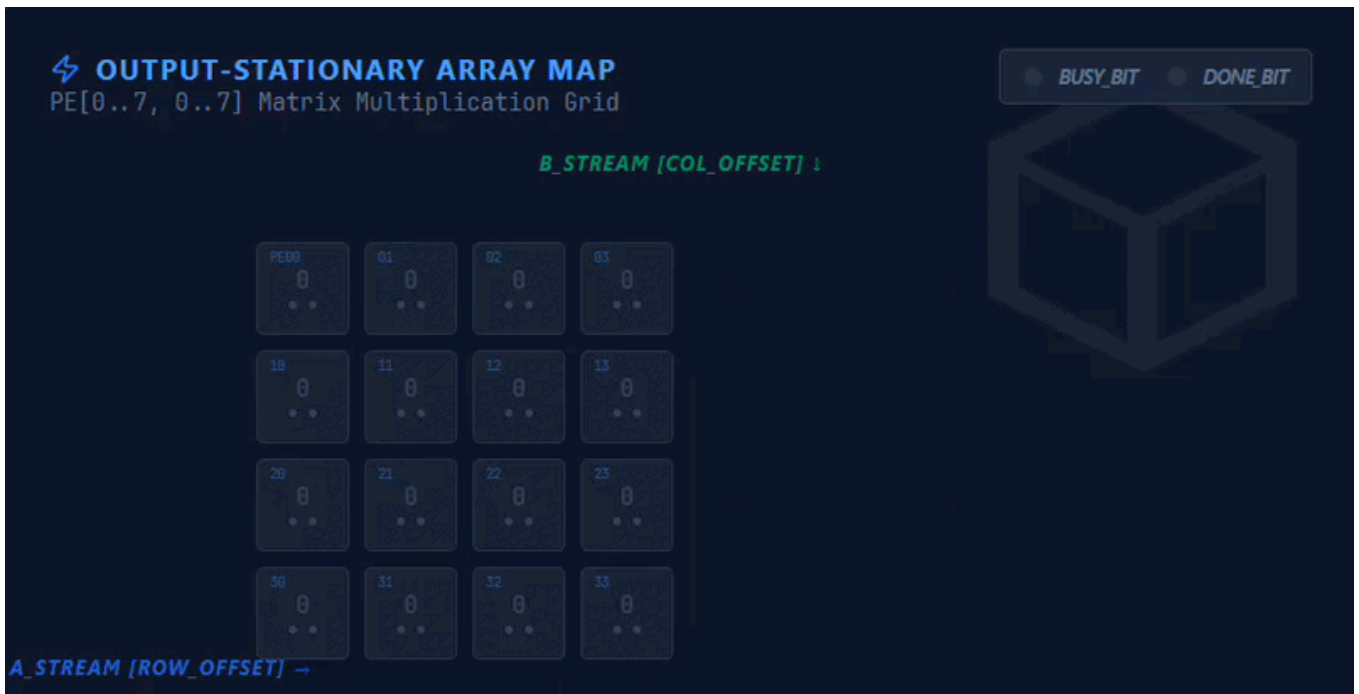
This approach is simple and useful for prototyping because the software can access accelerator registers directly. However, PIO communication becomes a system bottleneck when matrix data is repeatedly transferred between the HPS and FPGA. This limitation is confirmed in the measured results, where raw FPGA computation takes nanoseconds while total system transfers take hundreds of microseconds.

5. The FPGA performs matrix multiplication.
6. The HPS checks the accelerator status.
7. The HPS reads the output matrix.
8. The FPGA result is compared with a software reference result.
9. Execution time is measured and compared with the ARM NEON baseline.

JSON is used because it provides a portable and structured format for representing matrix data. The JSON standard defines it as a lightweight text-based data interchange format [5].

3. Hardware Architectures

3.1 Output-Stationary Systolic Array



The first architecture implemented in this project is an Output-Stationary Systolic Array. In this architecture, each processing element accumulates an output value. Input values move through the array while partial sums remain stationary inside the processing elements.

This approach reduces unnecessary movement of intermediate results and allows multiple multiply-and-accumulate operations to occur in parallel. Systolic architectures are designed around rhythmic data movement and repeated local processing, making them suitable for structured computations such as matrix multiplication [3].

The main advantage of the systolic-array design is its regular structure. This makes the architecture easier to map onto FPGA logic and easier to scale compared with irregular custom datapaths.

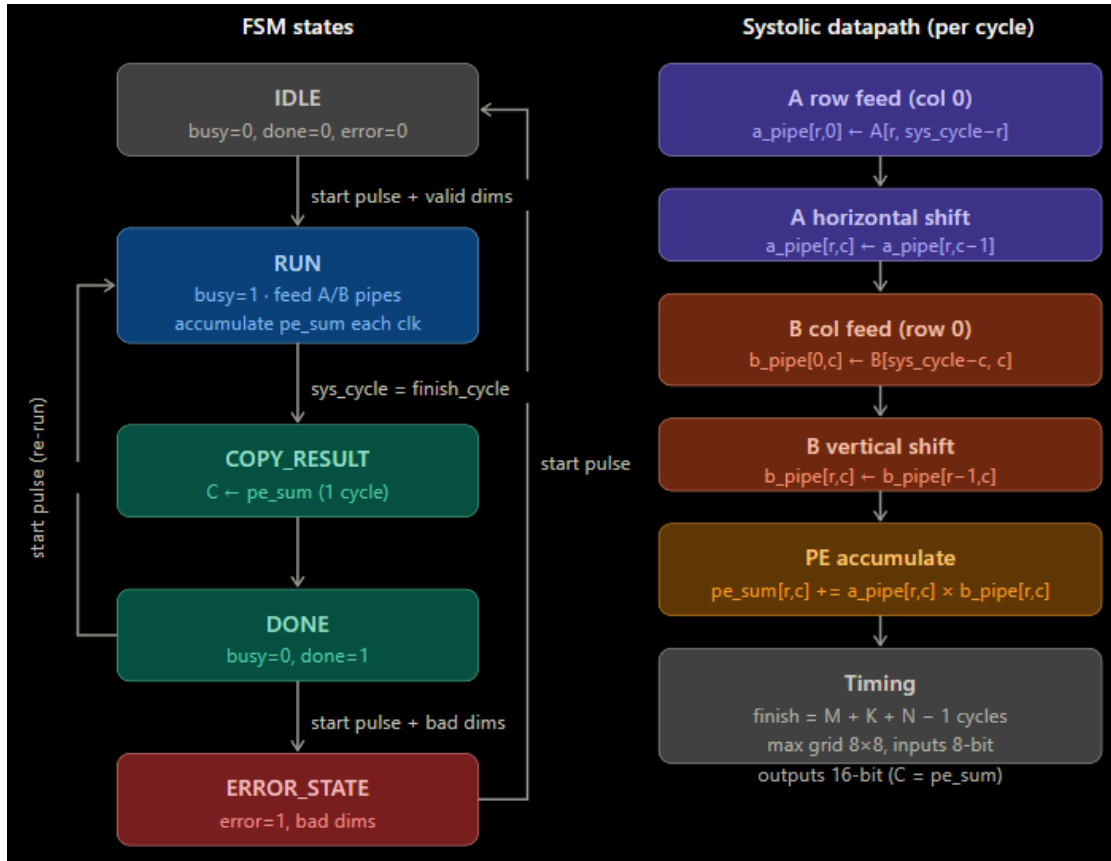


Figure 2. FSM and systolic datapath timing concept used in the accelerator design.

3.2 Output-Stationary Fully Parallel Architecture

The second architecture implemented in this project is the Output-Stationary Fully Parallel architecture. This design was used to investigate the maximum raw computation speed that can be achieved when more FPGA processing elements are used at the same time.

The reason for using this architecture is that the systolic array does not activate all processing elements immediately. In the systolic array, data enters gradually as a wavefront, so the first cycles use only a small number of PEs. In contrast, the fully parallel architecture tries to use all active output PEs from the first compute cycle. This reduces the number of computation cycles and gives higher raw throughput.

In this architecture for example $C[2 \times 2]$, each PE is assigned to one output element:

$PE[0][0] \rightarrow$ computes $C[0][0]$

$PE[0][1] \rightarrow$ computes $C[0][1]$

$PE[1][0] \rightarrow$ computes $C[1][0]$

$PE[1][1] \rightarrow$ computes $C[1][1]$

Each PE keeps its own partial sum locally. Therefore, it is still output-stationary because the output value does not move between PEs. The difference from the systolic array is how the input data reaches the PEs. Instead of shifting A and B through neighboring PEs, the fully parallel design directly provides the required A and B values to all active PEs at each k-step.

However, this performance comes at a cost. Since all PEs need access to the required A and B values every cycle, the design requires more routing and higher fanout. For example, $A[0][k]$ must be delivered to all PEs in row 0, and $B[k][0]$ must be delivered to all PEs in column 0. This makes the design faster in cycle count, but more expensive in FPGA routing and less scalable for larger matrix sizes.

The Fully Parallel architecture achieved the strongest raw computation performance in this project. It produced a measured 20.7x speedup compared with the ARM NEON-optimized software baseline. However, total system speedup was reduced when PIO communication overhead was included.

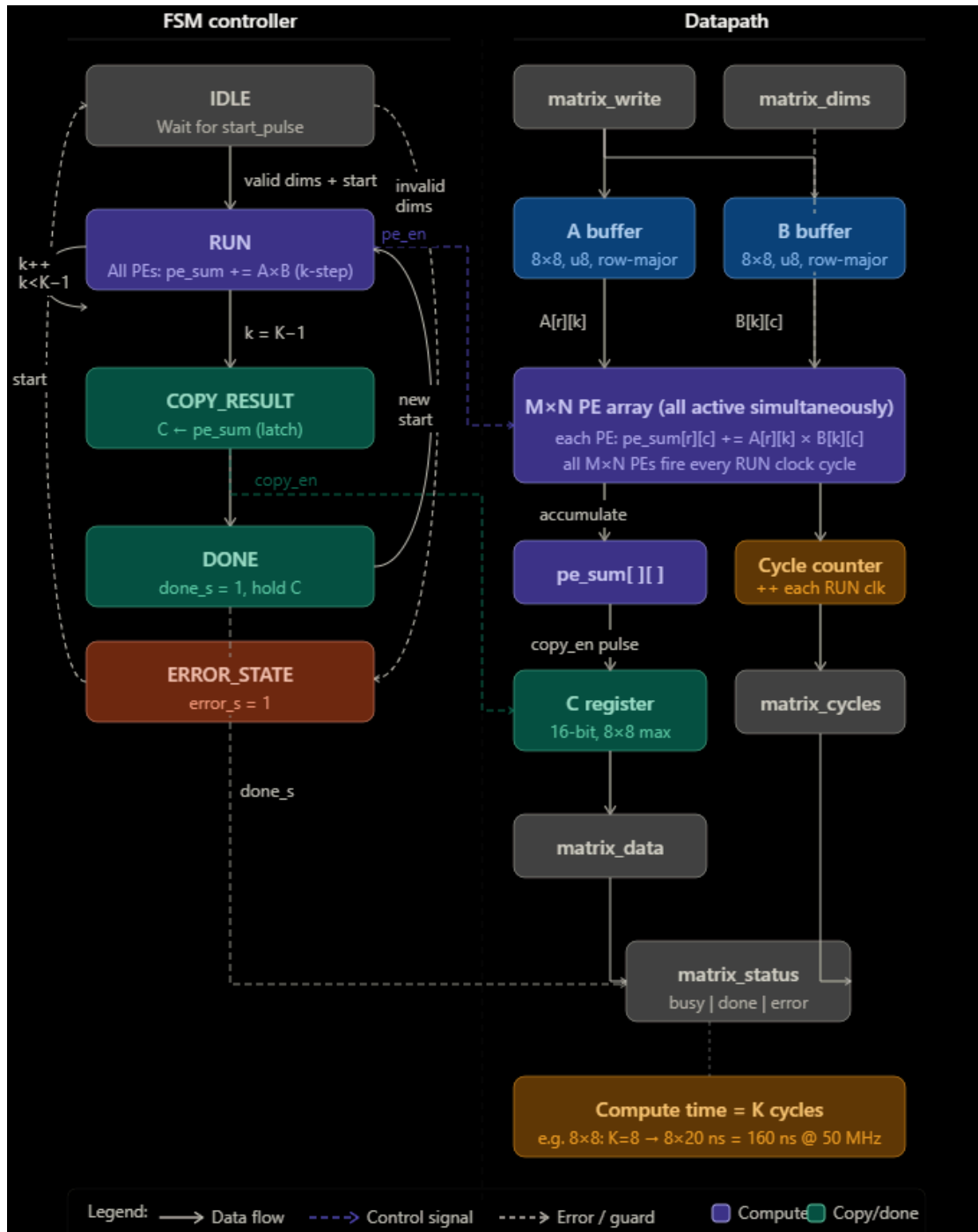


Figure 2. FSM and Full parallel datapath timing concept used in the accelerator design.

4. Software Implementation

The HPS software manages the complete execution flow. It reads matrix inputs from JSON files, validates matrix dimensions, writes values to memory-mapped PIO registers, starts the FPGA accelerator, waits for completion, and reads back the output matrix. In addition an ARM NEON-optimized implementation is used as the main software performance baseline. Arm provides matrix multiplication examples showing how NEON intrinsics can reduce arithmetic instruction count by using SIMD vector operations [4]. This makes ARM NEON a more meaningful comparison point than an unoptimized scalar C implementation.

The timing software reports both raw FPGA core cycles and total system time. Raw FPGA cycles are counted by hardware, while HPS-side transfer timings are collected using Linux `CLOCK_MONOTONIC`. The converted picosecond values in the logs are scaled from nanosecond timing and should not be interpreted as true picosecond measurement precision.

4.2 FPGA Control from HPS

The HPS communicates with the FPGA accelerator using memory-mapped Programmed I/O (PIO) registers. These registers are accessed from Linux using `/dev/mem` and `mmap()`.

The PIO interface allows the HPS software to write control information to the FPGA and read status and result information back.

The main PIO registers used by the software are shown in Table 1.

Table 1. PIO Registers Used for HPS–FPGA Communication

Register	Direction	Purpose
matrix_ctrl	HPS to FPGA	Starts the accelerator and selects the operation mode.
matrix_dims	HPS to FPGA	Stores matrix dimensions M, K, and N.
matrix_write_addr	HPS to FPGA	Selects the FPGA internal address for writing matrix data.
matrix_write_data	HPS to FPGA	Sends one matrix value to the FPGA.
matrix_write_en	HPS to FPGA	Generates a write pulse to store the input value.
matrix_status	FPGA to HPS	Indicates busy, done, error, and mode status.

matrix_index	HPS to FPGA	Selects which output matrix element to read.
matrix_data	FPGA to HPS	Returns the selected output matrix value.
matrix_cycles	FPGA to HPS	Returns the FPGA computation cycle count.

The matrix data is transferred using the address map shown in Table 2.

Table 2. Matrix Data Address Map

Address Range	Meaning
0–63	Matrix A storage locations
64–127	Matrix B storage locations

After matrix A, matrix B, and the matrix dimensions are written to the FPGA, the software sends a start pulse using the matrix_ctrl register. The FPGA then performs the computation and updates the status register when finished. The HPS reads the output matrix one element at a time using matrix_index and matrix_data.

4.1 ARM NEON Software Baseline

An ARM NEON-optimized software implementation was developed as the main software performance baseline. The purpose of this implementation is to compare the FPGA accelerator against an optimized software version running on the HPS processor.

NEON is the SIMD extension available in ARM processors. SIMD stands for Single Instruction Multiple Data, meaning that one instruction can operate on multiple data values at the same time. This makes NEON more efficient than basic scalar C for arithmetic-heavy operations such as matrix multiplication.

For each output element $C[i][j]$, matrix multiplication requires a dot product:

$$C[i][j] = A[i][0]B[0][j] + A[i][1]B[1][j] + \dots + A[i][K-1]B[K-1][j]$$

Since the maximum supported value of K is 8, the NEON implementation uses an 8-lane vector dot product. Matrix values are stored as 8-bit unsigned integers, and the accumulated result is stored as a 32-bit value. The NEON dot-product function loads 8 values from a row of matrix A and 8 values from a column of matrix B, then performs parallel multiplication using NEON intrinsics.

The main NEON operations used are shown in Table 3.

NEON Intrinsic	Function
vld1_u8()	Loads 8 unsigned 8-bit values into a NEON vector register.
vmull_u8()	Performs 8 parallel unsigned 8-bit multiplications and produces 16-bit products.
vmovl_u16()	Extends 16-bit products into 32-bit values.
vaddq_u32()	Adds 32-bit vector values.
vst1q_u32()	Stores vector values back to memory.

Table 3. Main NEON Intrinsics Used in the Software Baseline

Conceptually, the NEON dot product performs the following operation in parallel:

$$[A0\ A1\ A2\ A3\ A4\ A5\ A6\ A7] \times [B0\ B1\ B2\ B3\ B4\ B5\ B6\ B7]$$

This allows the ARM processor to compute multiple multiplication operations in one vector operation, making it a stronger and more meaningful software baseline than an unoptimized scalar C implementation.

4.2 Transposing Matrix B for NEON

In normal row-major memory storage, rows of matrix A are contiguous in memory, but columns of matrix B are not. However, NEON works most efficiently when the data to be loaded is stored contiguously. To solve this, the software transposes matrix B into a temporary matrix called B_T.

The transpose operation is:

$$B_T[c][r] = B[r][c]$$

After transposition, each original column of B becomes a contiguous row in B_T. This allows the NEON dot-product function to load both operands efficiently:

```
va = vld1_u8(A[row])
```

```
vb = vld1_u8(B_T[col])
```

This improves memory access efficiency and makes the HPS software baseline stronger and more representative of optimized software performance.

5. Verification and Testing

Correctness was verified by comparing the FPGA output matrix with the output of the software reference implementation. Several matrix dimensions were tested up to the supported maximum size of 8 x 8. The purpose of these tests was to confirm that the FPGA accelerator produced the same results as the software implementation.

For the 8 x 8 test case, the systolic FPGA design, the fully parallel FPGA design, and the ARM NEON software implementation produced the same output matrix. This confirms functional correctness for the main performance test case.

Performance testing was divided into two categories: raw computation time and total system time. Raw computation time measures only the time required by the FPGA accelerator to perform matrix multiplication. Total system time includes computation time and communication time between the HPS and FPGA. This distinction is important because an accelerator may compute quickly but still provide limited end-to-end speedup if data transfer overhead is high.

6. Results and Discussion

6.1 Timing Summary

Table 1 summarizes the measured raw computation times and speedups for the 8 x 8 matrix multiplication test. The FPGA clock frequency was 50 MHz, so one FPGA cycle equals 20 ns.

Implementation	FPGA cycles	Raw compute time	Measured system time	Raw speedup vs. ARM NEON
ARM NEON baseline	N/A	3308.672 ns	3308.672 ns per run	1.0x
Output-Stationary Systolic Array	24	480 ns	314.790 us	6.9x
Output-Stationary Fully Parallel	8	160 ns	327.510 us	20.7x

Table 1. Timing summary for 8 x 8 matrix multiplication.

6.2 Transfer Breakdown

Table 2 shows that the largest contributors to total FPGA system time are the PIO write transfers for matrices A and B and the PIO readback transfer for matrix C. These values are much larger than the raw FPGA core times, confirming that the communication path dominates the end-to-end latency.

Timing component	Systolic Array	Fully Parallel
A write time	127.620 us	126.530 us
B write time	127.590 us	123.790 us
Start overhead	1.960 us	1.940 us
HPS wait for done	1.110 us	1.120 us
C read time	50.720 us	49.490 us
Total FPGA system time	314.790 us	327.510 us

Table 2. HPS-FPGA transfer and control timing breakdown.

The measured HPS wait-for-done time was approximately the same for both architectures: 1.110 μ s for the OS systolic array and 1.120 μ s for the fully parallel output-stationary design. This is expected because the HPS

wait measurement includes software polling overhead, memory-mapped PIO read latency, and HPS–FPGA bridge latency. The difference in FPGA core computation time between the two architectures is only 320 ns, which is smaller than the processor-visible polling overhead. Therefore, the wait-for-done time cannot be used alone to compare the internal FPGA computation speed. The FPGA cycle counter provides the accurate core latency, while the HPS wait time reflects how quickly the Linux software can observe the done status through the PIO interface.

6.3 Correctness Output

For the 8 x 8 test case, all implementations produced the same result matrix shown below. This matching output was used to verify the FPGA designs against the software reference result.

1	2	3	4	5	6	7	8
224	179	203	215	208	195	171	160
271	221	246	261	249	238	209	201
318	263	289	307	290	281	247	242
365	305	332	353	331	324	285	283
412	347	375	399	372	367	323	324
459	389	418	445	413	410	361	365
506	431	461	491	454	453	399	406
553	473	504	537	495	496	437	447

Table 3. Verified output matrix $C = A \times B$ for the 8 x 8 test case.

6.4 Resource Mapping and Fabric Utilization

The physical implementation shows a clear contrast between the fully parallel and systolic architectures. The Fully Parallel design occupies a larger overall area and produces denser routing activity across the fabric. This is expected because the architecture broadcasts data to many processing elements simultaneously, increasing routing demand and switching activity.

In contrast, the Systolic Array shows a more localized footprint. Because data is passed locally between adjacent processing elements, the data flow reduces routing pressure and produces a more organized physical implementation. This suggests that the systolic architecture offers better resource efficiency and thermal behavior, even though the fully parallel design achieves lower raw compute latency.

Both designs show activity near the HPS interface because the processor must send matrix inputs to the FPGA fabric and retrieve the computed results. The fully parallel design displays more intense activity leading away from this interface, while the systolic design maintains a more localized signal path.

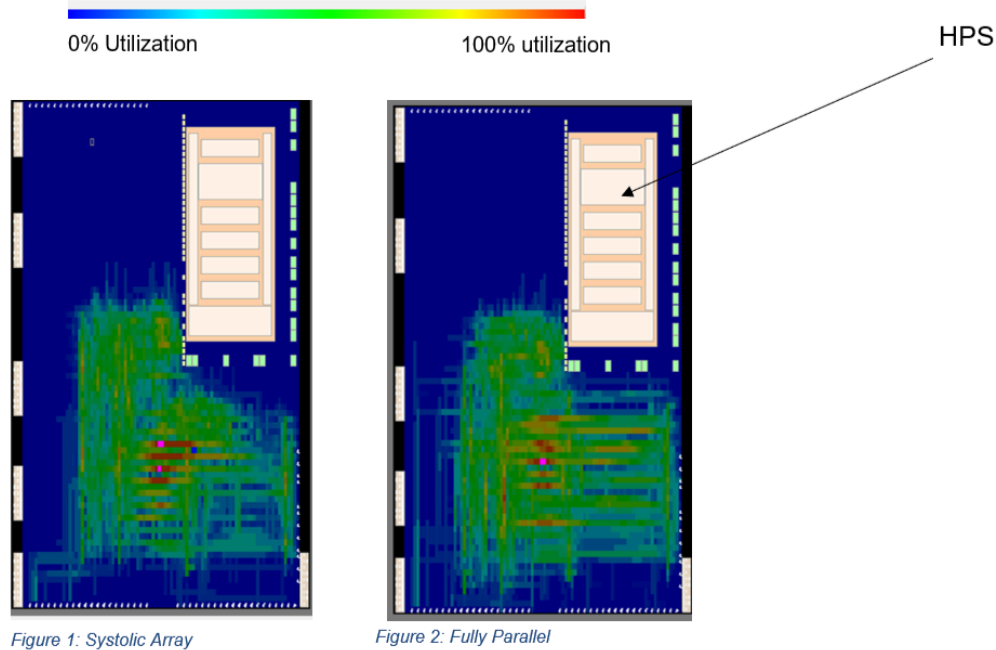


Figure 3. Fabric utilization comparison between the systolic-array and fully parallel designs.

6.5 Discussion

The raw FPGA computation results demonstrate that both hardware architectures accelerate the matrix multiplication kernel compared with the ARM NEON baseline. The systolic-array design reduces raw compute time to 480 ns, while the fully parallel design reduces it further to 160 ns.

However, total system timing shows that PIO communication dominates the end-to-end execution time. The FPGA core itself is not the performance bottleneck; instead, the largest delays come from writing matrix A, writing matrix B, and reading matrix C through the memory-mapped PIO interface.

This result highlights an important hardware/software co-design lesson: improving only the compute unit is not enough. To achieve strong end-to-end acceleration, the data transfer path must also be optimized. A future DMA-based design would reduce processor involvement, improve data movement efficiency, and allow the accelerator to better benefit from its fast computation core.

6.5.1 TimeQuest Timing Analysis and Architectural Trade-Off

For the **Output-Stationary Systolic Array**, TimeQuest reported:

Metric	Value
Clock	CLOCK_50
Fmax	59.08 MHz

Target clock	50 MHz
Timing result	Meets timing

This means the systolic design can safely run at the 50 MHz FPGA clock. Since the achieved Fmax is higher than 50 MHz, the design has timing margin.

For the **Output-Stationary Fully Parallel architecture**, TimeQuest reported:

Metric	Value
Clock	CLOCK_50
Fmax	53.77 MHz
Target clock	50 MHz
Timing result	Meets timing

This design also meets the 50 MHz requirement, but with less timing margin than the systolic design.

The trade-off is important:

Architecture	Advantage	Trade-Off
Output-Stationary Systolic Array	Better timing margin and local data movement	More cycles because the array fills and drains like a wavefront
Output-Stationary Fully Parallel	Lowest cycle count and fastest raw compute time	More routing pressure and lower Fmax

The fully parallel design is faster in cycle count because all active output PEs compute from the first compute cycle. However, this requires many operands to be distributed to many PEs at the same time. For example, each $A[\text{row}][k]A[\text{row}][k]A[\text{row}][k]$ value must be delivered to multiple PEs across a row, and each $B[k][\text{col}]B[k][\text{col}]B[k][\text{col}]$ value must be delivered to multiple PEs across a column. This creates higher fanout and more global routing.

The systolic architecture, on the other hand, moves data locally. Matrix AAA values shift horizontally, matrix BBB values shift vertically, and each PE mainly communicates with nearby PEs. This local movement reduces

routing complexity and explains why the systolic design achieved a higher Fmax of 59.08 MHz compared with 53.77 MHz for the fully parallel architecture.

Therefore, the fully parallel design wins in raw cycle latency, while the systolic design has better timing scalability and cleaner physical structure.

7. Conclusion

This project successfully designed, implemented, and evaluated a matrix multiplication accelerator on an Intel Cyclone V SoC FPGA platform. Two output-stationary hardware architectures were explored: a systolic-array architecture and a fully parallel architecture. The system supports flexible matrix dimensions up to 8 x 8 and uses JSON files on the Linux-based HPS to provide matrix inputs.

The Fully Parallel FPGA architecture achieved a 20.7x raw computation speedup over the ARM NEON-optimized software baseline, while the systolic-array design achieved approximately 6.9x speedup. These results confirm that FPGA-based parallel hardware can significantly accelerate the matrix multiplication computation itself.

However, total system performance was limited by PIO communication overhead. Since the lightweight HPS-to-FPGA bridge is more appropriate for control and status access than bulk data transfer, it is not ideal for moving matrix data repeatedly [1]. The main conclusion is that the FPGA accelerator is computationally effective, but future versions should replace PIO-based data movement with a DDR/DMA-based transfer approach.

8. Future Work

10. Replace PIO-based data transfer with DMA or shared DDR buffering.
11. Support matrix dimensions larger than 8 x 8.
12. Improve memory access efficiency and reduce software transfer overhead.
13. Add support for floating-point or configurable fixed-point arithmetic.
14. Develop a more complete Linux driver instead of relying only on direct memory-mapped access.
15. Compare performance against additional optimized software baselines.
16. Analyze FPGA resource utilization, timing closure, and power consumption in more detail.
17. Explore scalability for machine learning, signal processing, and embedded acceleration workloads.

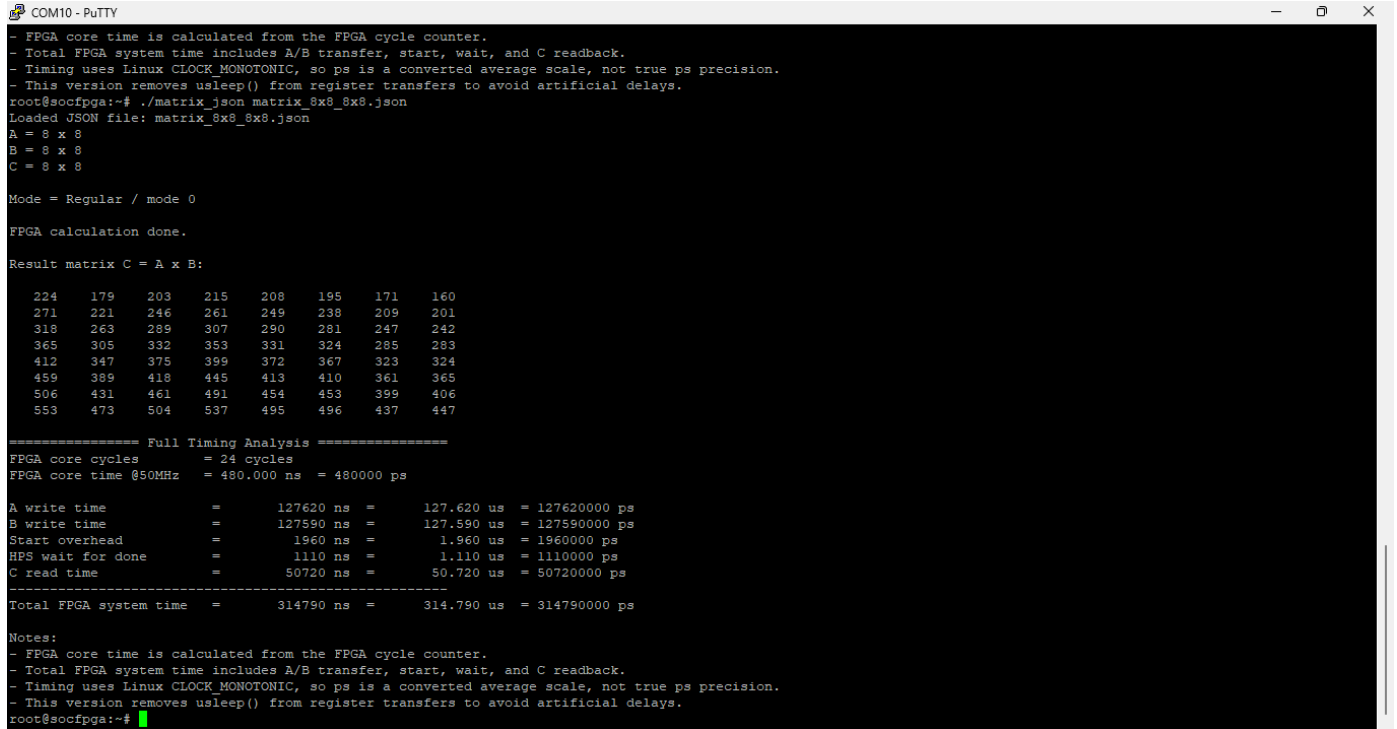
References

- [1] Intel Corporation, Cyclone V Hard Processor System Technical Reference Manual, Document ID 683126. Available: <https://www.intel.com/programmable/technical-pdfs/683126.pdf>
- [2] Intel Corporation, Avalon Interface Specifications, Document ID 683091. Available: <https://www.intel.com/programmable/technical-pdfs/683091.pdf>
- [3] H. T. Kung, "Why Systolic Architectures?," Computer, vol. 15, no. 1, pp. 37-46, 1982. Available: <https://www.eecs.harvard.edu/~htk/publication/1982-kung-why-systolic-architecture.pdf>
- [4] Arm Developer, "Example - Matrix Multiplication," Optimizing C code with Neon intrinsics. Available: <https://developer.arm.com/documentation/102467/0201/Example---matrix-multiplication>

- [5] T. Bray, Ed., The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259, IETF, Dec. 2017. Available: <https://datatracker.ietf.org/doc/html/rfc8259>

Appendix A. Timing Evidence

The following screenshots provide supporting evidence for the measured timing results reported in Section 6. The main body of the report uses summarized tables for clarity, while the screenshots preserve the original test outputs.



```
COM10 - PuTTY
- FPGA core time is calculated from the FPGA cycle counter.
- Total FPGA system time includes A/B transfer, start, wait, and C readback.
- Timing uses Linux CLOCK_MONOTONIC, so ps is a converted average scale, not true ps precision.
- This version removes usleep() from register transfers to avoid artificial delays.
root@socfpga:~# ./matrix_json matrix_8x8_8x8.json
Loaded JSON file: matrix_8x8_8x8.json
A = 8 x 8
B = 8 x 8
C = 8 x 8

Mode = Regular / mode 0

FPGA calculation done.

Result matrix C = A x B:

  224  179  203  215  208  195  171  160
  271  221  246  261  249  238  209  201
  318  263  289  307  290  281  247  242
  365  305  332  353  331  324  285  283
  412  347  375  399  372  367  323  324
  459  389  418  445  413  410  361  365
  506  431  461  491  454  453  399  406
  553  473  504  537  495  496  437  447

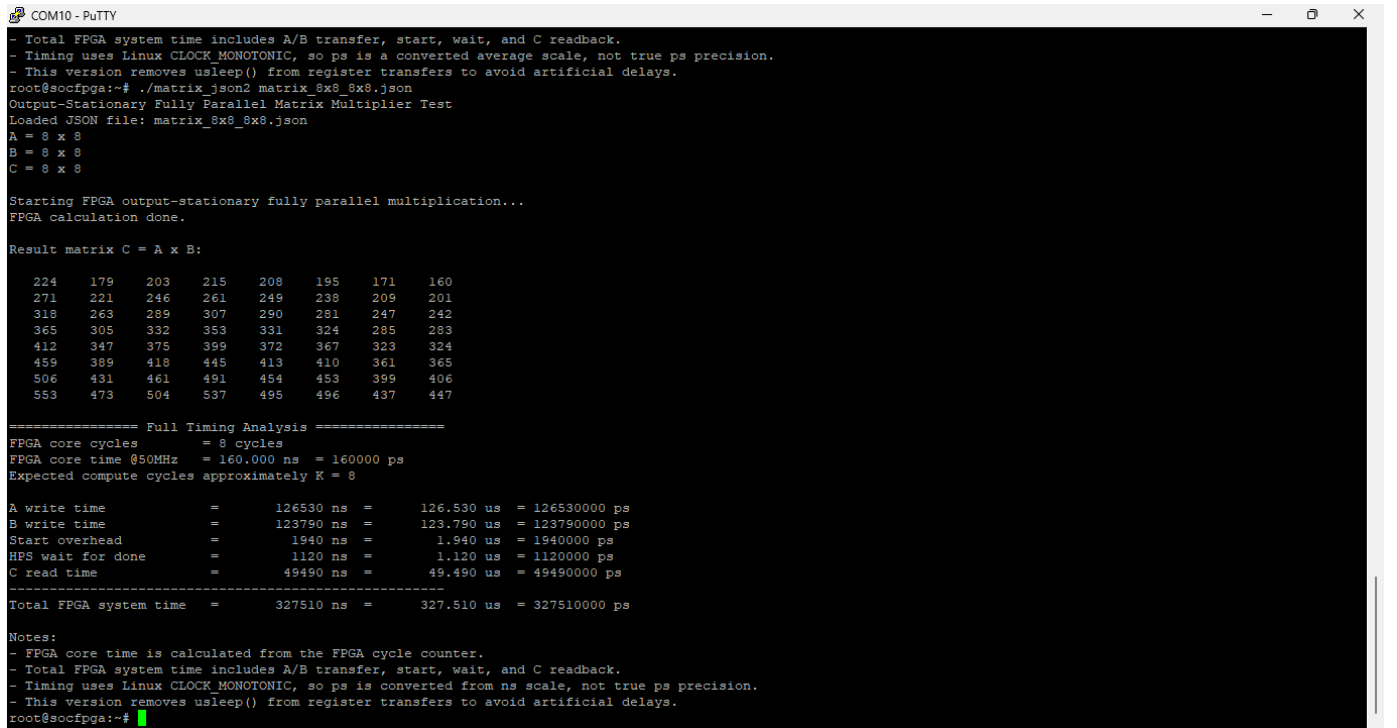
===== Full Timing Analysis =====
FPGA core cycles      = 24 cycles
FPGA core time @50MHz = 480.000 ns = 480000 ps

A write time         =      127620 ns =      127.620 us = 127620000 ps
B write time         =      127590 ns =      127.590 us = 127590000 ps
Start overhead       =       1960 ns =       1.960 us = 1960000 ps
HPS wait for done    =       1110 ns =       1.110 us = 1110000 ps
C read time          =       50720 ns =       50.720 us = 50720000 ps

-----
Total FPGA system time =      314790 ns =      314.790 us = 314790000 ps

Notes:
- FPGA core time is calculated from the FPGA cycle counter.
- Total FPGA system time includes A/B transfer, start, wait, and C readback.
- Timing uses Linux CLOCK_MONOTONIC, so ps is a converted average scale, not true ps precision.
- This version removes usleep() from register transfers to avoid artificial delays.
root@socfpga:~#
```

Figure A1. Systolic-array timing output.



```
COM10 - PuTTY
- Total FPGA system time includes A/B transfer, start, wait, and C readback.
- Timing uses Linux CLOCK_MONOTONIC, so ps is a converted average scale, not true ps precision.
- This version removes usleep() from register transfers to avoid artificial delays.
root@socfpga:~# ./matrix_json2 matrix_8x8_8x8.json
Output-Stationary Fully Parallel Matrix Multiplier Test
Loaded JSON file: matrix_8x8_8x8.json
A = 8 x 8
B = 8 x 8
C = 8 x 8

Starting FPGA output-stationary fully parallel multiplication...
FPGA calculation done.

Result matrix C = A x B:

  224  179  203  215  208  195  171  160
  271  221  246  261  249  238  209  201
  318  263  289  307  290  281  247  242
  365  305  332  353  331  324  285  283
  412  347  375  399  372  367  323  324
  459  389  418  445  413  410  361  365
  506  431  461  491  454  453  399  406
  553  473  504  537  495  496  437  447

===== Full Timing Analysis =====
FPGA core cycles      = 8 cycles
FPGA core time @50MHz = 160.000 ns = 160000 ps
Expected compute cycles approximately K = 8

A write time         =      126530 ns =      126.530 us = 126530000 ps
B write time         =      123790 ns =      123.790 us = 123790000 ps
Start overhead       =       1940 ns =       1.940 us = 1940000 ps
HPS wait for done    =       1120 ns =       1.120 us = 1120000 ps
C read time          =       49490 ns =       49.490 us = 49490000 ps

-----
Total FPGA system time =      327510 ns =      327.510 us = 327510000 ps

Notes:
- FPGA core time is calculated from the FPGA cycle counter.
- Total FPGA system time includes A/B transfer, start, wait, and C readback.
- Timing uses Linux CLOCK_MONOTONIC, so ps is converted from ns scale, not true ps precision.
- This version removes usleep() from register transfers to avoid artificial delays.
root@socfpga:~#
```

Figure A2. Fully parallel timing output.

```
COM10 - PuTTY
===== Full Timing Analysis =====
FPGA core cycles      = 8 cycles
FPGA core time @50MHz  = 160.000 ns = 160000 ps
Expected compute cycles approximately K = 8

A write time          = 126530 ns = 126.530 us = 126530000 ps
B write time          = 123790 ns = 123.790 us = 123790000 ps
Start overhead        = 1940 ns = 1.940 us = 1940000 ps
HPS wait for done     = 1120 ns = 1.120 us = 1120000 ps
C read time           = 49490 ns = 49.490 us = 49490000 ps

-----
Total FPGA system time = 327510 ns = 327.510 us = 327510000 ps

Notes:
- FPGA core time is calculated from the FPGA cycle counter.
- Total FPGA system time includes A/B transfer, start, wait, and C readback.
- Timing uses Linux CLOCK_MONOTONIC, so ps is converted from ns scale, not true ps precision.
- This version removes usleep() from register transfers to avoid artificial delays.
root@socfpga:~#
root@socfpga:~#
root@socfpga:~#
root@socfpga:~# ./matrix_json matrix_8x8_8x8.json
root@socfpga:~# ./hps_matrix_neon matrix_8x8_8x8.json
HPS ARM NEON Matrix Multiplication Benchmark
Loaded JSON file: matrix_8x8_8x8.json
A = 8 x 8
B = 8 x 8
C = 8 x 8
Iterations = 1000000

Result matrix C = A x B:

224 179 203 215 208 195 171 160
271 221 246 261 249 238 209 201
318 263 289 307 290 281 247 242
365 305 332 353 331 324 285 283
412 347 375 399 372 367 323 324
459 389 418 445 413 410 361 365
506 431 461 491 454 453 399 406
553 473 504 537 495 496 437 447

Timing result:
Total time          = 3308671870 ns
Average per run     = 3308.672 ns
Average per run     = 3308671.870 ps

Notes:
- This uses ARM NEON intrinsics for each dot product.
- B is transposed in software so NEON can load each B column contiguously.
- The ps value is converted from averaged ns timing; Linux is not truly ps-accurate.
root@socfpga:~#
```

Figure A3. Fully parallel timing output.

Appendix B. TimeQuest Analization

The screenshot displays the TimeQuest Timing Analyzer interface. The top menu bar includes File, View, Netlist, Constraints, Reports, Script, Tools, Window, and Help. The main window is titled "Slow 1100mV 85C Model". On the left, the "Set Operating Conditions" panel shows "Slow 1100mV 85C Model" selected. Below it, the "Report" panel shows "Fmax Summary" and "Report Timing" selected. The "Tasks" panel on the left lists various tasks like "Open Project...", "Netlist Setup", "Create Timing Netlist", "Read SDC File", "Update Timing Netlist", "Reset Design", "Set Operating Conditions...", "Reports", "Slack", "Report Setup Summary", "Report Hold Summary", "Report Recovery Summary", "Report Removal Summary", and "Report Minimum Delay Width". The main area shows a table of "Command Info" with columns: Slack, From Node, To Node, Launch Clock, Latch Clock, Relationship, Clock Skew, and Data Delay. The table contains one entry with a slack of 3.074 ns. Below this, the "Path #1: Setup slack is 3.074" section shows a detailed path summary with columns: Path Summary, Statistics, Data Path, Waveform, and Extra Filter Information. The path summary includes a table of "Data Arrival Path" and a table of "Data Required Path". The "Data Arrival Path" table has columns: Total, Incr, RF, Type, Fanout, Location, and Element. The "Data Required Path" table has columns: Total, Incr, RF, Type, Fanout, Location, and Element. The bottom of the interface shows a console window with the command: `report_timing -from_clock { CLOCK_50 } -to_clock { CLOCK_50 } -setup -npaths 1 -detail full_path -panel_name {Report Timing} -multi_corner` and the output: `Report Timing: Found 1 setup paths (0 violated). Worst case slack is 3.074`.

Figure B1. Platform Designer / system integration screenshot.

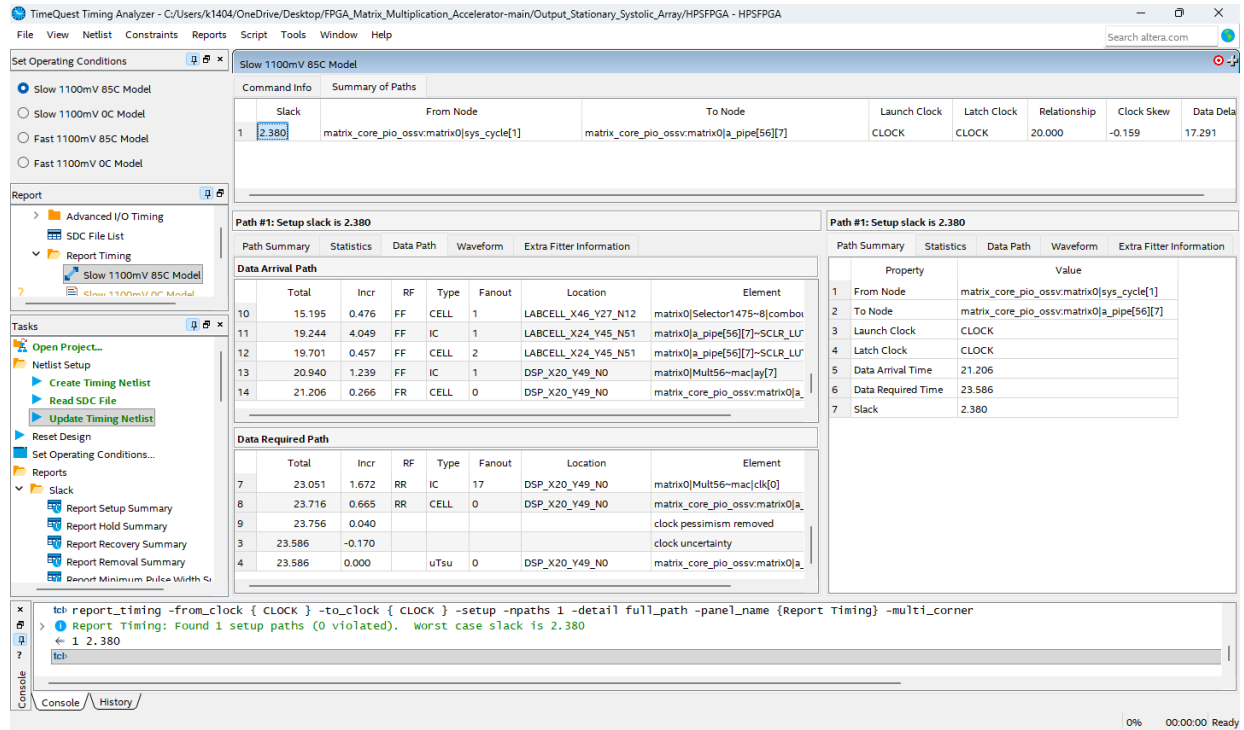


Figure B2. FPGA compilation and resource summary screenshot.

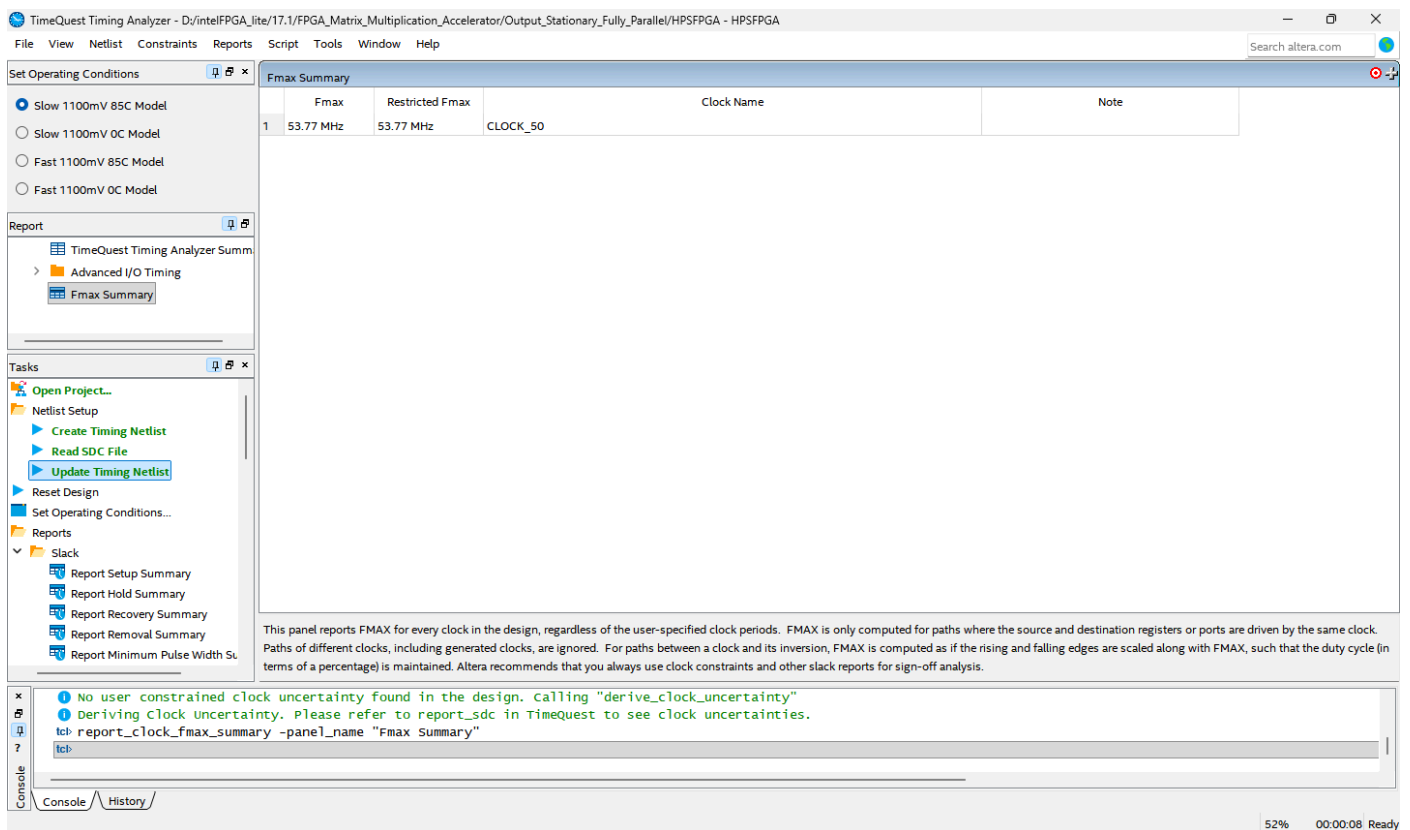


Figure B3. TimeQuest Analyzer for Full parallel design

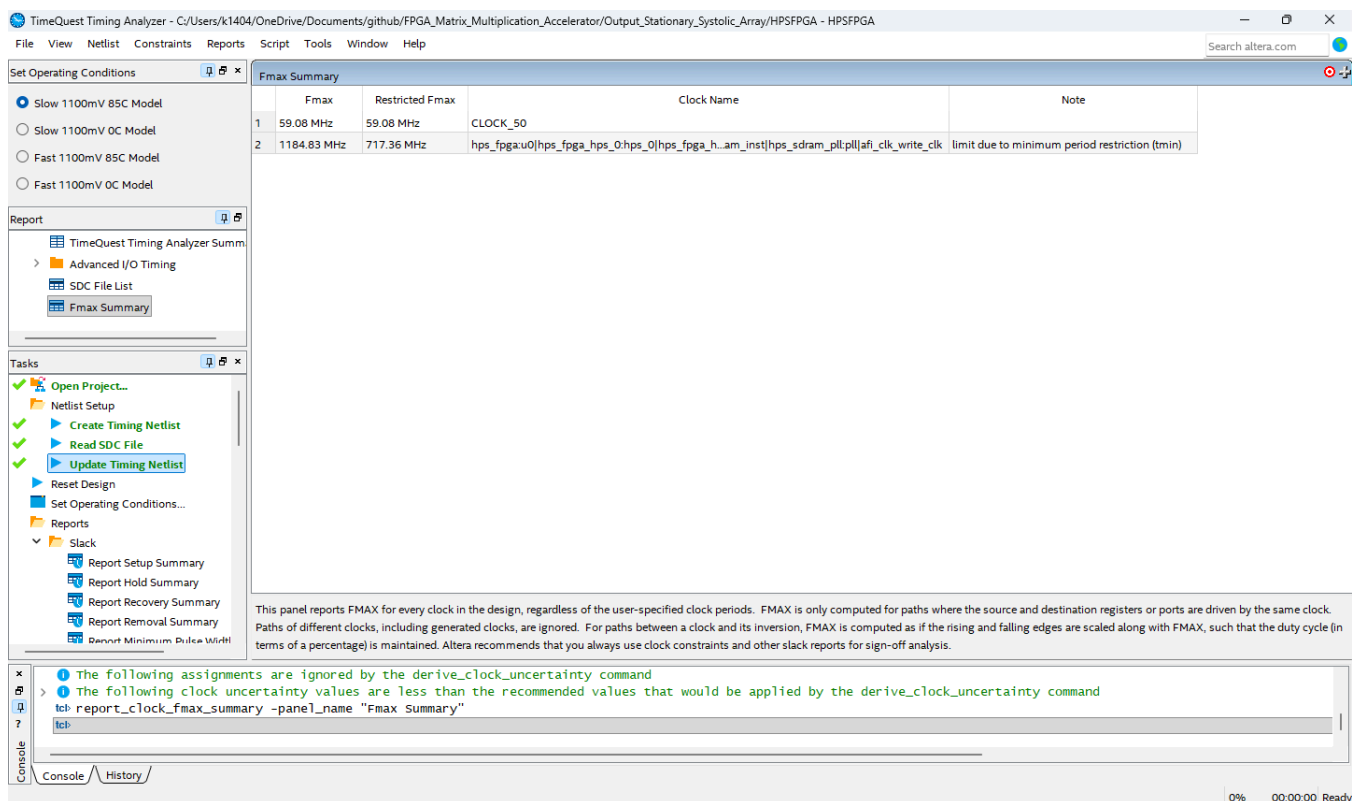


Figure B3. TimeQuest Analyzer for the Systolic design

Appendix A: Important Project Files

This appendix briefly explains the most important files used in the project.

A.1 Main Project Folders

Folder	Description
HSP_matrix_multi	Contains the HPS software implementations used as CPU-side baselines.
matrices	Contains JSON matrix input files used for testing different matrix sizes.
Output_Stationary_Systolic_Array	Contains the FPGA project and source files for the Output-Stationary Systolic Array architecture.
Output_Stationary_Fully_Parallel	Contains the FPGA project and source files for the Output-Stationary Fully Parallel architecture.

A.2 HPS Software Files

File	Location	Description
<code>hps_matrix.c</code>	<code>HSP_matrix_mu lti</code>	Regular HPS C matrix multiplication implementation. Used as a basic software baseline.
<code>hps_matrix_neon.c</code>	<code>HSP_matrix_mu lti</code>	ARM NEON-optimized matrix multiplication implementation. Used as the main optimized software performance baseline.
<code>hps_matrix</code>	<code>HSP_matrix_mu lti</code>	Compiled executable for the regular HPS software version.
<code>hps_matrix_neon</code>	<code>HSP_matrix_mu lti</code>	Compiled executable for the ARM NEON software benchmark.

A.3 Matrix Input Files

File	Location	Description
<code>matrix_8x8_8x8.json</code>	<code>matric es</code>	Main 8×8 test matrix used for performance comparison.
<code>matrix_2x3_3x2.json</code>	<code>matric es</code>	Small non-square test case used to verify flexible matrix dimensions.
<code>matrix_3x4_4x3.json</code>	<code>matric es</code>	Medium non-square test case.
<code>matrix_4x4_4x4.json</code>	<code>matric es</code>	Square 4×4 test case.
<code>matrix_tests_expected_results.txt</code>	<code>matric es</code>	Contains expected output results for matrix tests.

A.4 Output-Stationary Systolic Array Files

File	Location	Description
<code>matrix_core_pio_ossv.v</code>	<code>Output_Stationary_Systolic_Array</code>	Main Verilog HDL file for the Output-Stationary Systolic Array accelerator.
<code>matrix_json.c</code>	<code>Output_Stationary_Systolic_Array</code>	HPS C program used to control and test the systolic FPGA accelerator.
<code>matrix_json</code>	<code>Output_Stationary_Systolic_Array</code>	Compiled executable for testing the systolic accelerator on HPS Linux.
<code>HPSFPGA.v</code>	<code>Output_Stationary_Systolic_Array</code>	Top-level Verilog file connecting the HPS system with the FPGA matrix core.
<code>hps_fpga.qsys</code>	<code>Output_Stationary_Systolic_Array</code>	Platform Designer system file containing HPS, bridges, and PIO connections.
<code>hps_0.h</code>	<code>Output_Stationary_Systolic_Array</code>	Hardware address header used by the HPS C program to access PIO registers.

A.5 Output-Stationary Fully Parallel Files

File	Location	Description
<code>matrix_core_pio_output_stationaryv.v</code>	<code>Output_Stationary_Fully_Parallel</code>	Main Verilog HDL file for the Output-Stationary Fully Parallel accelerator.

<code>matrix_json2.c</code>	<code>Output_Stationary_Fully_Parallel</code>	HPS C program used to control and test the fully parallel FPGA accelerator.
<code>matrix_json2</code>	<code>Output_Stationary_Fully_Parallel</code>	Compiled executable for testing the fully parallel accelerator on HPS Linux.
<code>HPSFPGA.v</code>	<code>Output_Stationary_Fully_Parallel</code>	Top-level Verilog file connecting the HPS system with the fully parallel matrix core.
<code>hps_fpga.qsys</code>	<code>Output_Stationary_Fully_Parallel</code>	Platform Designer system file for the fully parallel architecture.
<code>hps_0.h</code>	<code>Output_Stationary_Fully_Parallel</code>	Hardware address header used by the HPS C program.

A.6 Important Quartus Files

File	Description
<code>HPSFPGA.qpf</code>	Quartus project file.
<code>HPSFPGA.qsf</code>	Quartus settings file containing device, pin, and source file assignments.
<code>HPSFPGA.sdc</code> / <code>HPSFPGA.out.sdc</code>	Timing constraint files used by TimeQuest.

`hps_fpga.sopcinfo`

Platform Designer system information file.